GSoC Proposal for AI Agent for Apache HertzBeat with MCP

Sarthak Arora

Abstract

This proposal outlines the development of an AI Agent for Apache HertzBeat using the **Model Context Protocol** (MCP), enabling natural language interaction with monitoring capabilities. The project will implement an MCP-compliant server and client architecture integrated with **SpringAI**, along with a chat-based frontend for the HertzBeat dashboard. This enhancement will make HertzBeat's monitoring features more accessible while serving as a reference implementation for AI integration in monitoring tools.

1 Personal Details

- Name: Sarthak Arora
- University: Birla Institute of Technology and Science, Pilani (2024 graduate)
- Email: sarthakeash@gmail.com
- GitHub: github.com/sarthakeash
- Instant Messaging: Whatsapp / Email
- Country of Residence: India
- Timezone: IST (GMT + 0530)
- Primary Language: English

2 Introduction

I am Sarthak Arora from India. I am a Software Engineer at a US based SaaS company Entelligence AI. I completed my **Computer Science** Engineering from BITS Pilani University, India in 2024. I have over 4 years of experience in building Full Stack Applications and AI agents. I am well versed in **Java**, **Typescript**, **React** and **Angular**. As a part of my course work, particularly Object Oriented Programming, and Object Oriented Analysis and Design, I have built projects using Java and **Spring Boot**. At my current company, I work with NextJS Typescript and Django backend, and have made **interactive Chat UIs**, integrated and made AI agents using **LangChain** and **Agentic frameworks**. I have also had experience with connecting external data sources with LLMs for live querying.
Hence, my academic and professional experience makes me a strong fit for the requirements of this project. As far as open source is concerned, I am quite new to it and applying to **GSoC** for the very **first time**. Recently, I got my first open source PR merged into HertzBeat, felt really great to contribute and be a part of it!
When I am not working, you can find me playing Lawn Tennis or my favourite video games. From time to time, I like to go trekking and explore beautiful landscapes.

3 Background

Apache HertzBeat provides comprehensive monitoring capabilities, alerts and connects with a variety of data sources. But it lacks the intuitive **natural language** interaction. With the rise in AI and LLMs, and those being integrated with the most basic to complex applications, having an **AI agent** that can execute all the features becomes the need of the hour.
Users must navigate traditional dashboards and forms to query monitoring data or configure new targets—a process that can be time-consuming and requires technical familiarity with the platform. This problem can be solved effectively by implementing a conversational agent, which will surely lead to wider adoption of HertzBeat.
All functions/APIs are already in place to set up monitoring targets, querying information, trigger alerts etc. We need to effectively curate on how to leverage these functions as tools for the LLM/AI agent to execute them based on the user's demands. Using **MCP as a communication protocol** is a really good approach. Implementing an MCP server also ensures that the users can access the data from

third-party apps like Cursor, WindSurf, Claude Desktop, etc. Moreover, the support for MCP servers is ever increasing.

4 Description of Work

4.1. Research and Architecture Design

1. Study HertzBeat architecture and codebase thoroughly
2. Research MCP implementation options with SpringAI
3. Configure and discover required dependencies. For example,
    - `spring-ai-mcp-server-spring-boot-starter` for mcp-server
    - `spring-ai-mcp-client-spring-boot-starter` for mcp-client
4. Note all the key operations to be exposed via the agent

4.2. Core AI Agent Development

4.2.1 MCP Server Service

1. Define new Data Structures and Object Classes for tools.
2. Implement MCP compliant tools structure, listing schemas for all the required functions or tools in the relevant classes. The key operations are:
    - Query monitor status
    - Retrieve alarm information
    - Search metrics data
    - Add new monitoring targets
    - Configure Thresholds for alarms

3. Aggregate all the tools from the various tool classes and use the **ToolCallbackProvider** class to expose them.

The following is an example code of how various classes can be structured with the respective tools in each of them. Spring AI uses descriptions for every method and param. Please note that the **attributes** passed in these methods are just for **illustration purposes**.

```java
public class MonitoringTools {

   @Tool(description = "Query the status of monitors. Can filter by ID or type.")
   public String queryMonitorStatus(
           @ToolParam(description = "ID of the specific monitor") Long monitorId,
           @ToolParam(description = "Type of monitor (website, database, server,
etc)") String monitorType) {
       return "Monitor status information would be returned here";
   }

   @Tool(description = "Add a new monitoring target to HertzBeat")
   public String addNewMonitor(
           @ToolParam(description = "Name for the monitor") String name,
           @ToolParam(description = "Type of monitor (website, database, server,
etc)") String type,
           @ToolParam(description = "Host address of the target") String host) {
       return "New monitor has been added successfully";
   }
}

public class AlertTools {

   @Tool(description = "Retrieve alarm information with optional filters")
   public String retrieveAlarmInformation(
           @ToolParam(description = "ID of the alarm") Long alarmId,
           @ToolParam(description = "Alarm severity (critical, major, minor,
warning)") String severity) {
       return "Alarm information would be returned here";
   }

   @Tool(description = "Configure threshold-based alarms for a specific monitor")
   public String configureThresholds(
           @ToolParam(description = "ID of the alert define") Long alertDefineId,
           @ToolParam(description = "Name of the metric to alert on") String
metricName,
           @ToolParam(description = "Threshold value") Double threshold) {
       return "Threshold configuration has been set successfully";
   }
}

public class MetricsTools {

   @Tool(description = "Search and retrieve metrics data")
   public String searchMetricsData(
           @ToolParam(description = "ID of the monitor") Long monitorId,
           @ToolParam(description = "Name of the metric to search for") String
metricName,
           @ToolParam(description = "Time range (e.g. 1h, 24h, 7d)") String
timeRange) {
       return "Metrics data matching the search criteria would be returned here";
   }
}
```

4. On Spring app startup, this Server will be exposed to hosts using **HTTP SSE API** at <port-number>/api/ai/mcp/sse
5. Check RBAC for this Server. Use sureness.yml file for deciding what all tools will be available to the users according to their role.

4.2.2 Implementation of Tools and Integration with HertzBeat Services

1. Existing Services, e.g. MonitorService, AlertDefineService, etc can be directly used in the implementation of the various tools.
2. String type to be returned for the primarily void functions, so that LLM can understand the outcome better, and frame the user response accordingly
3. As a pseudo implementation to demonstrate the use of existing HertzBeat service I have implemented the relevant methods if user wants to add a **new monitoring target**

```
class MonitoringToolsImpl implements MonitoringTools{
private AppServiceImpl appService;
private MonitorService monitorService;
 // monitor dto, similar DTOs to be defined for Hierarchy

@JsonIgnoreProperties(ignoreUnknown = true)
public record MonitorDto(
   @JsonProperty("monitor") Monitor monitor,
   @JsonProperty("params") List<Param> params,
   @JsonProperty("collector") Collector collector,
   @JsonProperty("grafanaDashboard") GrafanaDashboard grafanaDashboard
) {
   @JsonIgnoreProperties(ignoreUnknown = true)
   public record Monitor(
       @JsonProperty("id") Long id,
       @JsonProperty("name") String name,
       @JsonProperty("app") String app,
       @JsonProperty("host") String host,
       @JsonProperty("status") Integer status
   ) {}

   @JsonIgnoreProperties(ignoreUnknown = true)
   public record Param(
       @JsonProperty("field") String field,
       @JsonProperty("value") String value
   ) {}

   @JsonIgnoreProperties(ignoreUnknown = true)
   public record Collector(
       @JsonProperty("id") Long id,
```

```
        @JsonProperty("name") String name
    ) {}

    @JsonIgnoreProperties(ignoreUnknown = true)
    public record GrafanaDashboard(
        @JsonProperty("id") Long id,
        @JsonProperty("name") String name
    ) {}
}

@Tool("Fetch all apps")
public List<Hierarchy> getAllAppsTool(String userLang) {
    return appService.getAllAppHierarchy(userLang);
}

@Tool("Fetch all params for an app")
public List<ParamDefine> getAppParamDefinesTool(String app) {
    return appService.getAppParamDefines(app);
}

@Tool("Detect monitor")
public String detectMonitorTool(MonitorDto monitorDto) {
        //validation checks to be added
try{
monitorService.detectMonitor(monitorDto.getMonitor(), monitorDto.getParams(),
monitorDto.getCollector());
return "Monitor Detection Success";
}catch (Exception e){
return "Monitor could not be detected" +e.getMessage() ;
}
}

@Tool("Add monitor if successful detection")
public String addMonitorToolAfterSuccess(MonitorDto monitorDto) {
try{
monitorService.addMonitor(monitorDto.getMonitor(), monitorDto.getParams(),
monitorDto.getCollector(),monitorDto.getGrafanaDashboard());
return "Monitor added"
} catch (Exception e){
Return "Monitor addition failed" +e.getMessage();
}
}


@Tool("Delete or save monitor")
void deleteOrSaveMonitorToolBasedOnInput(boolean isDelete, MonitorDto monitorDto) {
 if (isDelete) {
           // do nothing
 } else {
try{
  monitorService.addMonitor(monitorDto.getMonitor(), monitorDto.getParams(),
monitorDto.getCollector(), monitorDto.getGrafanaDashboard());
```

```
return "Monitor added"
 }}catch(Exception e){
return "Could not be added" +e.getMessage();
}
}
}
```

4.2.3 MCP Client and AI Agent Service

1. Make an MCP client service class that connects with the server and accesses all the available tools.
2. Expose the method that can be used by the Chat Controller to connect to this client and pass in the user queries
3. Add history to the user query for additional context using the Conversation Service
4. Add relevant system prompts. This also helps in optionally structuring the data. For example, if we want metrics data to always return in the form of a chart, we can mention that. Or conditionally append the same to the prompt if certain tools are decided to be called by LLM.
5. Add the tool calling and execution logic via the Agent Service, in integration with the chosen LLM provider i.e executing the tool and giving the response back to LLM.
6. LLM knows about all the tools, and can decide which tools to call in what order according to the parameters and user query.
7. If LLM feels that it lacks certain information based on the parameters of the tools, it can revert back to ask the user for more and specific information. Or link them to read the user guide for obtaining keys etc.

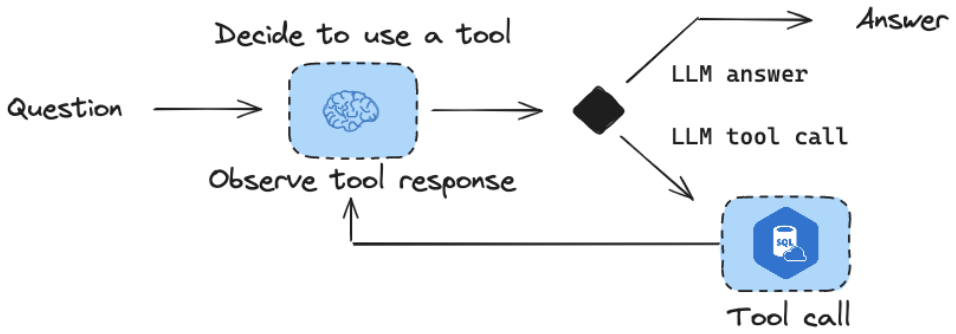

Figure 1: LLM interaction with tools

4.3. Controller Classes and DAO Development

1. Develop REST APIs so that the frontend can interact with the AI agent.
2. APIs reside inside the Controller class and serve as the communication medium between MCP client and the Frontend
3. Implement Chat Streaming HTTP endpoints to stream real time response
4. Create a basic Database Schema and Table in the common module
5. Design DAOs that implement the JPARepository interface
6. The following APIs (or Methods) to be included:

REST API Endpoints

- `POST /api/chat/stream`
- `GET /api/chat/history/{sessionId}`
- `GET /api/chat/conversations`
- `POST /api/chat/conversation`
- `DELETE /api/chat/conversation/{sessionId}`
- `POST /api/chat/setApiKey`

Helper Methods that connect with DAO

- `createNewConversation()`
- `getConversationMessages(String sessionId)`
- `saveConversation(String sessionId, List<Message> messages)`

4.4 Frontend UI Development

1. Create a new route inside the app folder for the Chat UI
2. Create a chat component with message list and input textarea
3. Integrate **Chat Stream** APIs
4. Add Option in settings or in the Chat UI to let the user select their LLM and plug in the relevant API key. (or optionally fetch from application properties)
5. Handle State Changes/Features such as “New Chat”, “History”, “Clear Chat”, “Load Old Chats”

6. Create components for displaying metrics data as Charts, Graphs using libraries such as Chart.js, Mermaid and creating detection and parsing logic for the same
7. [Optional] Make Specialized components for frequently accessed data like server load information for better visual appeal
8. [Optional] Support voice to text using Speech Recognition APIs
9. [Optional] Support **Commands** like '/setAlerts', '/showMonitoringData' , etc. Some modifications will be needed in the AI agent service as well but it can make the chat more appealing and can potentially reduce the number of LLM calls.

4.5. Testing and Integration

1. Implement proper error handling and validations throughout the project
2. Implement Unit Tests for all the tool services
3. Test Integration with all the supported services in HertzBeat

4.6. Configuring Builds, Documentation and Release

1. Configure build process for the new module as well as the MCP server
2. Add user guide/ release-log for the AI agent features and capabilities
3. List out all the requirements and process for smooth MCP server integration with other hosts
4. Before finalizing, a beta version can be released for the Apache Dev community. The feedback can be inculcated accordingly.

5 Sequence Diagram for the System

I have prepared a detailed sequence diagram illustrating the communication flow between different components of the proposed AI Agent system for an example user query. This diagram visualizes how user requests flow through the frontend, MCP client, MCP server, and HertzBeat services, highlighting the role of each component in the architecture. Due to its comprehensive nature, the sequence diagram was too large to fit within this proposal document, but it is accessible at this link.
This visual representation provides a clear picture of the system's architectural design and helps demonstrate the technical feasibility of the proposed solution.

6 Proposed Frontend UI

The images below show a sample design for the HertzBeat AI Agent chat interface. This is just a basic idea or suggestion of how the chat UI might look. The final design will likely change based on community feedback, testing and demand of features. These mockups show how users might chat with the AI to check monitoring data, view alerts, and set up new monitoring targets. Some of the features shown are optional, and their feasibility is likely to be decided by how much time is left.

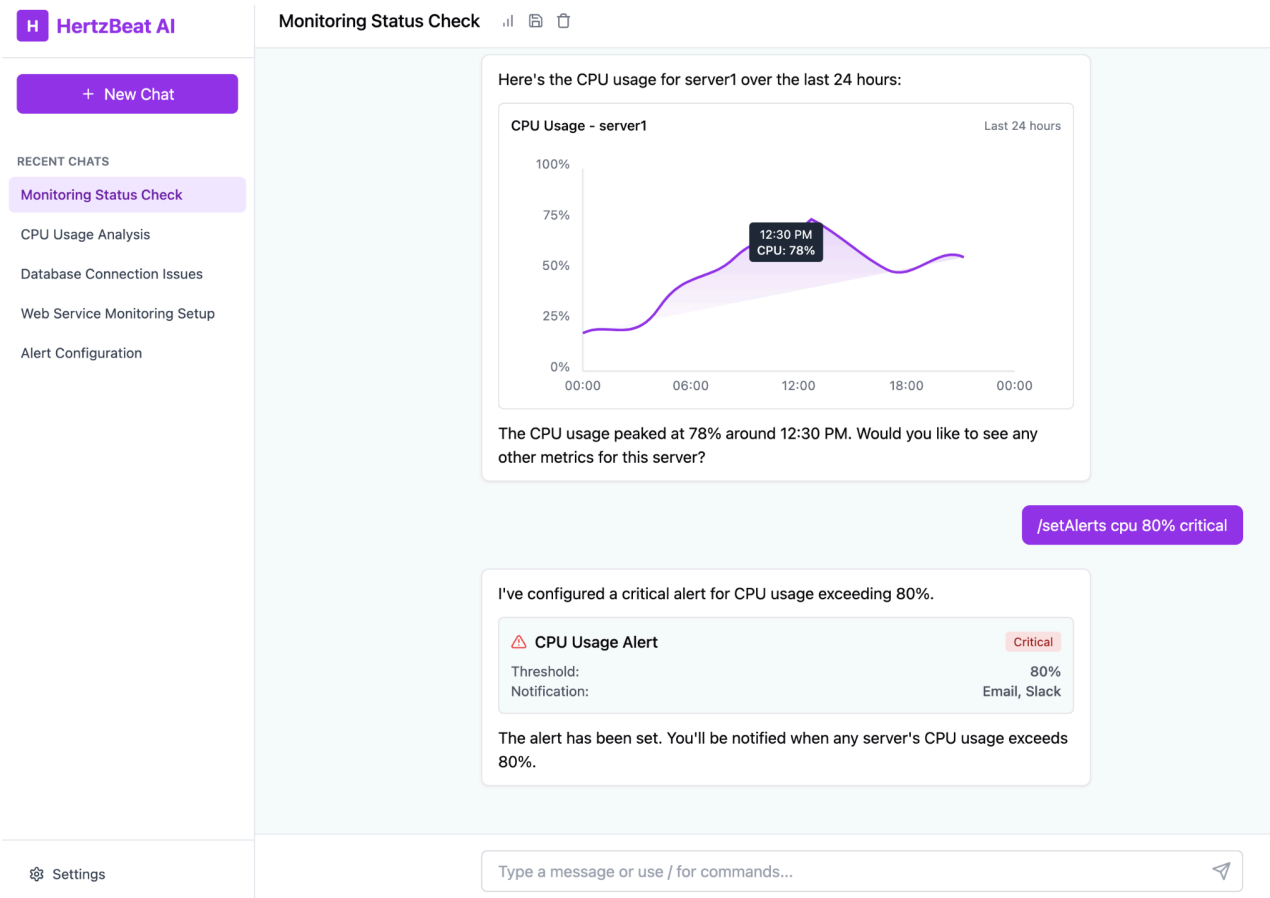

Figure 2: Main Chat UI with optional custom components

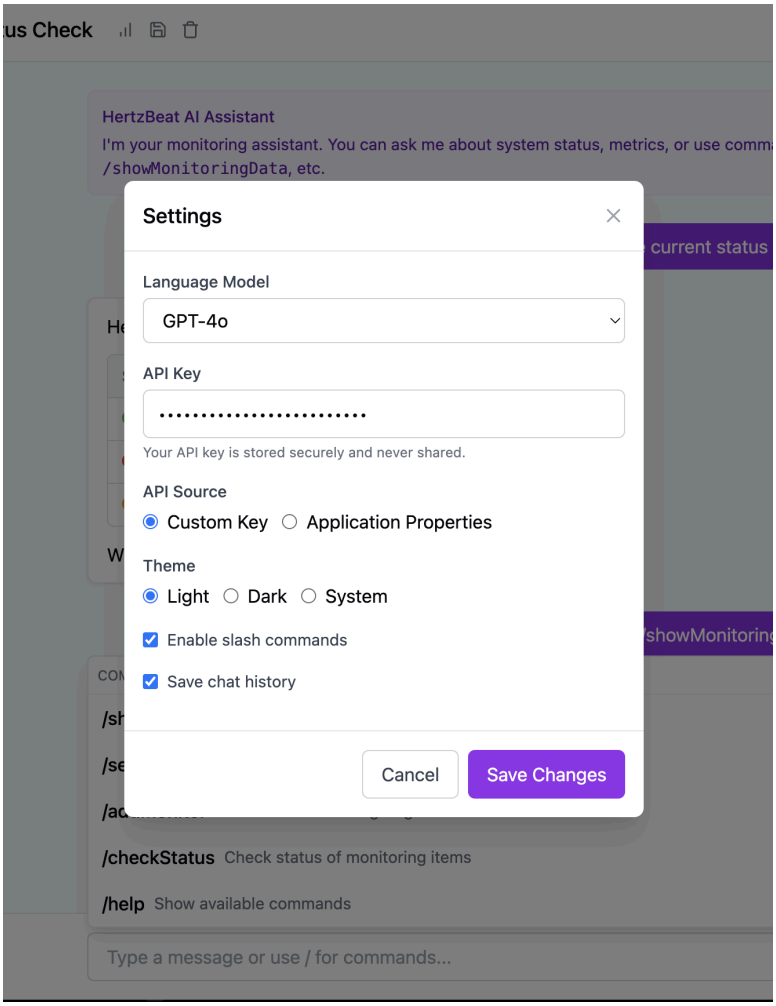

Figure 3: Settings Popup to change certain configurations

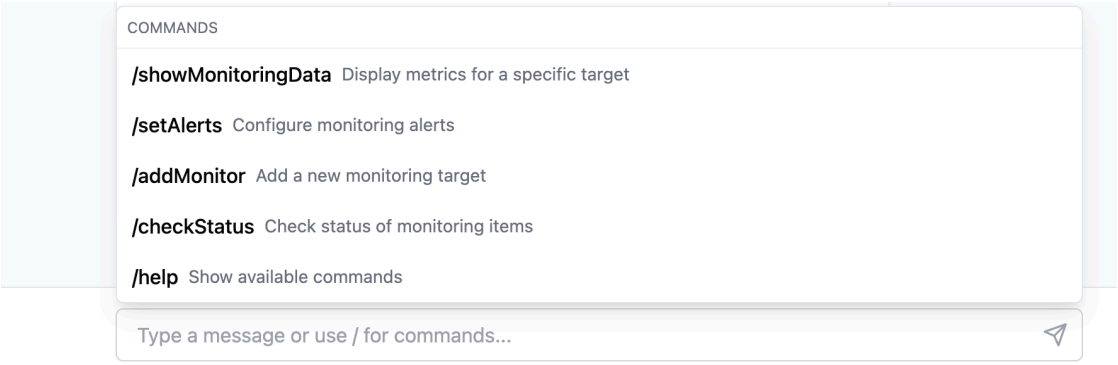

Figure 4: [Optional] Commands to easily route user queries

7 Backend Module Structure (Tentative)

```
hertzbeat-ai-agent/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── org/apache/hertzbeat/ai/
│   │   │       ├── AgentApplication.java
│   │   │       ├── config/
│   │   │       │   ├── McpServerConfig.java
│   │   │       │   ├── LlmConfig.java
│   │   │       │   ├── McpClientConfig.java
│   │   │       ├── controller/
│   │   │       │   ├── ChatController.java
│   │   │       │   └── ConversationController.java
│   │   │       ├── service/
│   │   │       │   ├── AgentService.java
│   │   │       │   ├── ConversationService.java
│   │   │       │   ├── McpClientService.java
│   │   │       │   ├── McpServerService.java
│   │   │       │   └── LlmProviderService.java
│   │   │       ├── service/impl/
│   │   │       │   ├── AgentServiceImpl.java
│   │   │       │   ├── ConversationServiceImpl.java
│   │   │       │   ├── McpClientServiceImpl.java
│   │   │       │   ├── McpServerServiceImpl.java
│   │   │       │   ├── AnthropicServiceImpl.java
│   │   │       │   └── OpenAIProviderServiceImpl.java
│   │   │       ├── tools/
│   │   │       │   ├── MonitoringTools.java
│   │   │       │   ├── AlertTools.java
│   │   │       │   └── MetricsTools.java
│   │   │       └── tools/impl/
│   │   │       │     ├── MonitorToolsImpl.java
│   │   │       │     ├── AlertToolsImpl.java
│   │   │       │     └── MetricsToolsImpl.java
│   │   │       ├── dao/
│   │   │       │   ├── ConversationDAO.java
│   │   │       │   ├── MessageDAO.java
│   │   │       │   └── UserPreferenceDAO.java
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── application-server.properties
│   │       ├── application-client.properties
│   │       └── prompts/
│   │           └── system-prompt.txt
└── pom.xml
```

* MCP Client and server directories could be separated with slight modifications

8 Results for the Apache community

This project will deliver significant value to the Apache community by introducing conversational AI capabilities to HertzBeat, making monitoring more accessible and efficient for users of all technical levels. The solution will reduce operational overhead, speed up troubleshooting, and promote HertzBeat to an even wider range of audience. Moreover, it will serve as a starting point for other users who might want to develop further on the intersection of monitoring and AI.

9 Deliverables

1. AI agent integrated with HertzBeat
2. Chat-based frontend component for the HertzBeat dashboard
3. MCP server for connection with external Hosts
4. Comprehensive test suite for all components
5. User and developer documentation

10 Timeline

10.1 Week 1

Study HertzBeat's code, research MCP options, and create design diagrams. Document key features and finalize tech choices after community discussions.

10.2 Week 2 - 3

Build the MCP server, create data models/DTOs, and design interfaces. Connect to HertzBeat services and develop tools for monitoring, alerts, and metrics.

10.2 Week 4 - 5

Develop the MCP client and connect to the server. Establish the LLM interaction with all the tools, conversation and context awareness.

10.3 Week 6 - 7

Create API endpoints. Call MCP client service, support streaming and history retrieval and passing. Create Database interactions and the relevant entities.

10.4 Week 8 - 9

Build a chat interface to display messages with streaming support and do conversation management. Match HertzBeat's design style and include LLM configuration and user preference settings.

10.5 Week 10 - 11

Write tests, check performance, and fix bugs. Use community feedback to improve the system before final integration.

10.6 Week 12

Create user guides, make demo videos, and clean up code. Address final feedback and prepare for submission. Implement Optional features if time is left.

11 Community Engagement

1. Regular updates on the dev mailing list of Apache (and/or HertzBeat)
2. Attending community meets for continuous feedbacks
3. User testing sessions with community members or mentors
4. Keeping an eye on new issues, see if something can be solved in this project's scope.
5. Engaging beyond GSoC by remaining active in the community and solving issues.

12 References

1. URL: *https://spring.io/projects/spring-ai*
2. URL: *https://docs.spring.io/spring-ai/reference/api/mcp/mcp-overview.html*
3. URL: *https://github.com/modelcontextprotocol/java-sdk*
4. URL: *https://modelcontextprotocol.io/introduction*

** All supplementary materials for this proposal—including **mockup images**, **sequence diagram**, and the proposed **directory structure** with sample files —are available in this GitHub repository: github.com/sarthakeash/hertzbeat-assets.